

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG**

ĐỀ CƯƠNG HỌC MÃ NGUỒN APP PYTHON

Học phần: Nhập môn Trí tuệ nhân tạo

Lớp: 22DTV_CLC1 - Nhóm: 05

**APP PYTHON YOLO + OCR NHẬN DẠNG BIỂN SỐ
PHƯƠNG TIỆN**

Sinh viên	: Lương Hải Long
MSSV	: 22207056
File mã nguồn chính	: AppPythonYOLO_OCR/app python vip pro kkkkkjsadljk.py
Vai trò liên quan	: YOLO, OCR, app Python/Android; báo cáo Typst/PDF, slide và điều phối đóng gói sản phẩm

Cách dùng tài liệu này

Tài liệu này được soạn để học mã nguồn app Python trước khi vận đáp. Mỗi phần giải thích app làm gì, hàm nào phụ trách, biến nào quan trọng, luồng chạy đi qua đâu và nên trả lời thế nào nếu giảng viên hỏi vào code.

Thành phố Hồ Chí Minh, tháng 5 năm 2026

MỤC LỤC

1	Bản Học Thuộc Nhanh	4
1.1	Nói gọn app Python trong 30 giây	4
1.2	Sơ đồ luồng chạy tổng quát	4
1.3	Các thư viện chính	4
2	Cấu Trúc File Mã Nguồn	6
2.1	Các vùng code lớn	6
2.2	Output của app nằm ở đâu?	6
2.3	Định dạng input	6
3	Preset Và Cấu Hình Suy Luận	8
3.1	Preset là gì?	8
3.2	Ý nghĩa các tham số trong preset	8
3.3	Cách trả lời về preset	8
4	Xử Lý Chuỗi Biểu Số	10
4.1	clean_text	10
4.2	fix_digits và fix_letters	10
4.3	valid_plate	10
4.4	normalize_plate	10
4.5	looks_like_parking_area_code	11
4.6	reliability_score	11
5	OCR Wrapper	13
5.1	Vì sao cần wrapper?	13
5.2	FastALPROCR.__init__	13
5.3	_predict_raw	13
5.4	_enhance	13
5.5	_two_line_candidates	13
5.6	FastALPROCR.read	14
5.7	FastPlateOCRWrapper	14
6	GUI Và Điều Phối Chạy	15
6.1	App.__init__	15
6.2	Các biến quan trọng trong __init__	15
6.3	build_ui	15
6.4	cfg	16
6.5	load_model và load_ocr	16
6.6	start và run	16
7	Chế Độ Xử Lý Ảnh	18

7.1	collect_images	18
7.2	run_images	18
7.3	log_image_results	18
8	Chế Độ Xử Lý Video	20
8.1	Vì sao video phức tạp hơn ảnh?	20
8.2	run_video tổng quan	20
8.3	sample_fps và stride	20
8.4	FFmpeg decode/encode	20
8.5	choose_video_encoder	21
8.6	mux_audio_from_source	21
9	Hàm detect_frame - Trái Tim Của App	22
9.1	Vai trò	22
9.2	Luồng xử lý của detect_frame	22
9.3	make_inference_variants	22
9.4	predict_boxes_on_image	23
9.5	dedup_detection_boxes	23
9.6	pass_geometry_filter	23
9.7	classical_plate_fallback_boxes	23
9.8	reject_by_anti_sign_guard	23
10	Tracking Và OCR Voting Trong Video	25
10.1	Vì sao cần tracking?	25
10.2	bbox_iou và ema_box	25
10.3	add_ocr_vote	25
10.4	update_tracks	25
10.5	tracks_to_results	26
11	Lọc Kết Quả Và Vẽ Output	27
11.1	clean_display_results	27
11.2	draw	27
11.3	show_frame và show_crop_preview	27
12	Bảng Học Từng Hàm Chính	28
12.1	Nhóm xử lý biên số và box	28
12.2	Nhóm OCR và app	28
12.3	Nhóm xử lý ảnh/video	29
13	Câu Hỏi Vắn Đáp Về App Python	30
13.1	Câu hỏi tổng quan	30
13.2	Câu hỏi về YOLO trong app	30

13.3	Câu hỏi về OCR	30
13.4	Câu hỏi về video	31
13.5	Câu hỏi về lỗi và giới hạn	31
14	Kịch Bản Mở Code Khi Vấn Đáp	32
14.1	Thứ tự trình bày khi mở file app	32
14.2	Đoạn nói mẫu 1 phút	32
15	Kết Luận Học Thuộc	33
15.1	Ba câu quan trọng nhất	33

1. Bản Học Thuộc Nhanh

1.1. Nói gọn app Python trong 30 giây

App Python là phần mềm desktop dùng để demo pipeline nhận dạng biển số ngoài notebook. Người dùng chọn model YOLO .pt, chọn ảnh, thư mục ảnh hoặc video. App nạp YOLO để phát hiện bounding box biển số, crop vùng biển số, đưa crop vào OCR để đọc ký tự, chuẩn hóa chuỗi, lọc kết quả nhiễu, vẽ bounding box lên ảnh/video và lưu kết quả ra thư mục output, crop và CSV.

Câu cần học thuộc

“Notebook chứng minh mô hình train được, còn app Python chứng minh mô hình dùng được trong demo. App nói các bước YOLO detect, crop biển số, OCR, lọc kết quả, vẽ ảnh/video và xuất CSV.”

1.2. Sơ đồ luồng chạy tổng quát

Bước	Hàm / phần code	Ý nghĩa
1	App.__init__, build_ui	Tạo cửa sổ Tkinter, biến điều khiển, nút RUN/STOP, bảng kết quả và preview.
2	choose_model, choose_file, choose_folder	Cho người dùng chọn model .pt, ảnh/video hoặc thư mục ảnh.
3	start, run	Bắt đầu xử lý bằng thread riêng, nạp model YOLO và OCR backend.
4	run_images hoặc run_video	Rẽ nhánh theo input là ảnh/folder ảnh hay video.
5	detect_frame	Chạy YOLO trên frame/ảnh, crop biển số, OCR và tạo PlateResult.
6	clean_display_results draw	Lọc kết quả cuối, chống trùng vùng, vẽ box và text lên ảnh/video.
7	log_image_results update_tracks	Ghi CSV đối với ảnh; với video thì tracking và OCR voting để ổn định kết quả.

Bảng 1: Luồng chạy tổng quát của app Python

1.3. Các thư viện chính

Thư viện	Dùng để làm gì	Câu nói khi bị hỏi
ultralytics	Nạp YOLO .pt, chạy model.predict để phát hiện biển số.	Đây là phần detector AI.
opencv-python / cv2	Đọc ảnh, crop, xử lý ảnh, vẽ bounding box, đọc/ghi frame.	OpenCV xử lý ảnh và video ở mức pixel.
numpy	Xử lý mảng ảnh và frame raw video.	Ảnh trong OpenCV là mảng numpy.
pandas	Ghi bảng kết quả ra CSV.	CSV giúp kiểm tra lại kết quả nhận dạng.
Pillow	Hiển thị ảnh lên canvas Tkinter.	Tkinter cần chuyển ảnh sang ImageTk.PhotoImage.
fast-alpr, fast-plate-ocr	OCR biển số từ crop.	Hai backend OCR dùng chung interface read(crop).
tkinter	Xây dựng giao diện desktop.	Đây là GUI cho thao tác demo.
subprocess, FFmpeg	Decode/encode video, mux audio.	FFmpeg giúp xuất video kết quả ổn định.

Bảng 2: Các thư viện chính trong requirements.txt và mã nguồn

2. Cấu Trúc File Mã Nguồn

2.1. Các vùng code lớn

File app có thể học theo bảy vùng lớn:

1. Import, đường dẫn output, cấu hình màu và đuôi file.
2. Preset cấu hình suy luận.
3. Hàm xử lý chuỗi biến số, bounding box và confidence.
4. Lớp OCR wrapper.
5. Lớp App(tk.Tk) xây dựng GUI và điều phối chạy.
6. Các hàm xử lý ảnh/video, fallback, tracking, OCR voting.
7. Hàm hiển thị, vẽ kết quả và mở output.

Câu hỏi: Nếu thay đổi file app tổ chức thế nào?

Trả lời mẫu: Em sẽ chia file thành ba lớp chính: lớp tiện ích xử lý biến số và bounding box, lớp OCR wrapper để đọc crop biến số, và lớp App để tạo giao diện, nạp model, xử lý ảnh/video, tracking, vẽ kết quả và xuất CSV.

2.2. Output của app nằm ở đâu?

Các biến đầu file tạo thư mục output:

```
OUT_DIR = APP_DIR / "outputs_clean"
IMG_ANNOT_DIR = OUT_DIR / "annotated_images"
CROP_DIR = OUT_DIR / "crops"
CSV_PATH = OUT_DIR / "results.csv"
CONFIG_PATH = OUT_DIR / "config.json"
```

Ý nghĩa:

- OUT_DIR: thư mục gốc chứa kết quả.
- IMG_ANNOT_DIR: ảnh đã vẽ box và text.
- CROP_DIR: crop biến số đã cắt từ ảnh/video.
- CSV_PATH: bảng kết quả nhận dạng.
- CONFIG_PATH: lưu cấu hình lần chạy trước để lần sau mở app không cần chọn lại hết.

2.3. Định dạng input

App phân biệt input bằng đuôi file:

- IMAGE_EXTS: ảnh như .jpg, .jpeg, .png, .bmp, .webp, .tif, .tiff.
- VIDEO_EXTS: video như .mp4, .mkv, .webm, .mov, .avi, .m4v, .ts, .mts, .m2ts.

Trong run, nếu input là thư mục hoặc ảnh thì gọi run_images; nếu là video thì gọi run_video.

3. Preset Và Cấu Hình Suy Luận

3.1. Preset là gì?

PRESETS là một dictionary chứa nhiều cấu hình chạy sẵn. Mỗi preset phù hợp với một tình huống demo. App có các preset:

- **Balanced:** cân bằng tốc độ và độ ổn định.
- **Image / Multi plates:** ảnh bãi xe, nhiều biển nhỏ.
- **Low confidence rescue:** cứu trường hợp YOLO bỏ sót vì confidence thấp.
- **Fast video:** video ưu tiên tốc độ.
- **Slow video:** video chậm hơn nhưng ổn định hơn.

3.2. Ý nghĩa các tham số trong preset

Tham số	Ý nghĩa cần học
sample_fps	Video không nhất thiết detect mọi frame. Tham số này quyết định số frame mỗi giây được đưa vào YOLO để tiết kiệm thời gian.
conf	Ngưỡng confidence của YOLO. Thấp thì bắt được nhiều box hơn nhưng dễ nhiễu; cao thì sạch hơn nhưng dễ bỏ sót.
imgsz	Kích thước ảnh đầu vào của YOLO. Lớn hơn có thể thấy biển nhỏ tốt hơn nhưng chạy chậm hơn.
smooth	Hệ số làm mượt box trong video bằng EMA.
hold_s	Thời gian giữ track sau khi tạm mất detection, giúp text không nhấp nháy.
max_show	Số biển tối đa hiển thị sau khi lọc.
min_reliability	Ngưỡng điểm tin cậy tổng hợp để giữ kết quả cuối.
only_valid_vn	Nếu bật, chỉ giữ chuỗi giống định dạng biển số Việt Nam.
hide_unknown	Ẩn kết quả không OCR được.
rotate_fallback	Chạy thêm ảnh xoay 90 độ để cứu biển nghiêng/dọc.
gamma_fallback	Làm sáng ảnh tối bằng gamma để YOLO/OCR dễ xử lý hơn.
tile_fallback	Chia ảnh thành nhiều tile để tìm biển nhỏ trong ảnh đông xe.
classical_fallback	Dùng xử lý ảnh cổ điển để đề xuất box khi YOLO bỏ sót.
anti_sign_guard	Lọc các nhãn nền như D28, D29 trong bãi xe để tránh nhận nhầm thành biển số.
crop_pad	Nới box trước khi crop để không mất ký tự ở mép biển.

Bảng 3: Ý nghĩa các tham số preset trong app

3.3. Cách trả lời về preset

Câu hỏi: Vì sao cần nhiều preset?

Trả lời mẫu: Vì mỗi loại input có yêu cầu khác nhau. Ảnh nhiều xe cần imgsz lớn, confidence thấp và tile scan để bắt biển nhỏ. Video cần cân bằng tốc độ nên có preset Fast video và Slow video. Preset giúp demo nhanh mà vẫn cho phép chỉnh tay nếu ảnh khó.

4. Xử Lý Chuỗi Biển Số

4.1. clean_text

```
def clean_text(s) -> str:
    return re.sub(r"^[A-Z0-9]", "", str(s or "").upper())
```

Hàm này đưa chuỗi về chữ hoa và loại tất cả ký tự không phải chữ/số. Ví dụ 64-K8.9380 thành 64K89380. Đây là bước nền cho mọi hàm kiểm tra biển số.

4.2. fix_digits và fix_letters

fix_digits dùng bảng DIGIT_FIX để sửa các ký tự dễ nhầm sang số, ví dụ 0 -> 0, I -> 1, S -> 5, B -> 8. fix_letters làm ngược lại trong phần cần chữ, ví dụ 0 -> 0, 1 -> I, 5 -> S.

Ý nghĩa: OCR có thể nhầm chữ và số. App không sửa bừa toàn bộ chuỗi, mà sửa theo ngữ cảnh: đoạn nào cần số thì dùng fix_digits, đoạn nào cần chữ thì dùng fix_letters.

4.3. valid_plate

```
VN_PLATE_RE = re.compile(r"^\d{2}[A-Z]{1,2}\d{4,6}$")
VN_MOTO_SERIES_RE = re.compile(r"^\d{2}[A-Z]\d\d{4,5}$")
```

```
def valid_plate(t) -> bool:
    t = clean_text(t)
    return (bool(VN_PLATE_RE.fullmatch(t)) or
            bool(VN_MOTO_SERIES_RE.fullmatch(t))) and t[:2] in VN_PROVINCE_CODES
```

Hàm này kiểm tra chuỗi có giống biển số Việt Nam hay không. Điều kiện gồm:

- bắt đầu bằng 2 chữ số mã tỉnh;
- mã tỉnh nằm trong VN_PROVINCE_CODES;
- sau đó là chữ cái/series và cụm số;
- khớp regex xe hơi hoặc xe máy.

4.4. normalize_plate

normalize_plate(raw) là hàm quan trọng nhất trong nhóm xử lý chuỗi. Hàm nhận chuỗi OCR thô, tạo nhiều candidate khác nhau rồi chọn candidate tốt nhất.

Luồng chính:

1. Gọi clean_text(raw).
2. Nếu chuỗi quá ngắn thì trả lại ngay.
3. Thử nhiều cách cắt chuỗi để tìm 2 số đầu là mã tỉnh.

4. Sửa phần mã tỉnh về số bằng `fix_digits`.
5. Sửa phần series về chữ bằng `fix_letters`.
6. Sửa phần số cuối về số bằng `fix_digits`.
7. Tạo candidate dạng xe một dòng hoặc xe máy hai dòng.
8. Candidate nào `valid_plate` được cộng điểm rất cao.
9. Chọn candidate có điểm tốt nhất.

Câu học thuộc về `normalize`

“`normalize_plate` không chỉ xóa dấu cách. Nó thử tái cấu trúc chuỗi OCR theo format biển số Việt Nam: mã tỉnh, series chữ, phần số. Candidate hợp lệ được ưu tiên cao hơn, nhờ đó giảm lỗi OCR nhầm O/0, I/1, S/5, B/8.”

4.5. `looks_like_parking_area_code`

Hàm này dùng để chặn các mã nền như D28, D29, B12, P3. Trong ảnh bãi xe, các ký hiệu cột hoặc ô đậu thường có màu và hình dạng giống chữ trên biển, dễ bị OCR đọc nhầm.

Logic chính:

- Nếu chuỗi đã là biển hợp lệ thì không chặn.
- Nếu chuỗi ngắn từ 1 đến 5 ký tự thì dễ là mã nền.
- Nếu không bắt đầu bằng 2 số mã tỉnh thì chặn.
- Nếu 2 số đầu không nằm trong mã tỉnh Việt Nam thì chặn.

4.6. `reliability_score`

```
score = 0.50 * ocr + 0.33 * det
if valid_plate(t):
    score += 0.12
if len(t) in (7, 8, 9):
    score += 0.05
```

Điểm `reliability_score` là điểm tin cậy thực dụng để demo, không phải xác suất khoa học tuyệt đối. Nó kết hợp:

- OCR confidence: trọng số 0.50.
- YOLO detection confidence: trọng số 0.33.
- bonus nếu chuỗi là biển số hợp lệ.
- bonus nhỏ nếu độ dài thường gặp của biển số.

Nếu chuỗi không hợp lệ, điểm bị giới hạn tối đa. Nếu detection confidence quá thấp, điểm cũng bị giới hạn. Điều này giúp app không hiển thị quá tự tin với kết quả yếu.

Câu hỏi: Trust/reliability trong app có phải xác suất đúng tuyệt đối không?

Trả lời mẫu: Không. Đây là điểm tin cậy thực dụng để demo, kết hợp confidence OCR, confidence YOLO và việc chuỗi có giống biên số Việt Nam hay không. Nó giúp lọc và xếp hạng kết quả, nhưng không phải xác suất được hiệu chuẩn khoa học.

5. OCR Wrapper

5.1. Vì sao cần wrapper?

App hỗ trợ hai OCR backend: FastALPR DefaultOCR và fast_plate_ocr. Hai thư viện có cách gọi khác nhau, nhưng app muốn phần còn lại chỉ gọi một interface chung:

```
text, ocr_conf, source = ocr.read(crop)
```

Vì vậy code tạo hai wrapper:

- FastALPROCR
- FastPlateOCRWrapper

Cả hai đều có hàm `read(crop_bgr)` trả về text, confidence và source.

5.2. FastALPROCR.__init__

Hàm khởi tạo kiểm tra thư viện fast-alpr có tồn tại không. Nếu có, nó tạo DefaultOCR với model cct-s-v2-global-model và device auto.

Ý cần nói: OCR được nạp một lần, không nạp lại mỗi ảnh. Điều này giúp app chạy nhanh hơn.

5.3. _predict_raw

Hàm `_predict_raw` gọi OCR thật sự:

1. Nếu crop rỗng thì trả chuỗi rỗng.
2. Gọi `self.ocr.predict(img_bgr)`.
3. Lấy text từ các field có thể có: text, plate, ocr.
4. Lấy confidence từ confidence, conf, score.
5. Nếu confidence là list hoặc numpy array thì lấy trung bình.
6. Trả về `clean_text(raw)` và confidence.

5.4. _enhance

Hàm `_enhance` tăng chất lượng crop bằng ảnh xám, CLAHE và sharpen. Mục tiêu là cứu các crop mờ/tối để OCR đọc lại lần hai.

Không phải lúc nào enhance cũng tốt, nên app giữ cả kết quả OCR gốc và kết quả OCR sau enhance, sau đó chọn candidate tốt nhất.

5.5. _two_line_candidates

Biển xe máy Việt Nam thường hai dòng. Hàm `_two_line_candidates` tách crop thành phần trên và phần dưới:

- Phần trên tìm header như 29Y1, 61T3.
- Phần dưới tìm cụm số.
- Ghép header + số thành candidate.
- Candidate hợp lệ được cộng điểm.

Ý nghĩa: Nếu OCR toàn crop không đọc đúng biển hai dòng, app thử đọc từng nửa ảnh để tăng cơ hội nhận đúng.

5.6. FastALPROCR.read

Đây là hàm OCR chính. Luồng:

1. OCR crop gốc.
2. OCR crop đã enhance.
3. Thêm candidate biển hai dòng.
4. Nếu không có candidate thì trả rỗng.
5. Chọn candidate tốt nhất theo thứ tự ưu tiên:
 - có phải biển hợp lệ không;
 - điểm candidate;
 - độ dài chuỗi;
 - OCR confidence.

Câu hỏi: OCR trong app đọc một lần hay nhiều lần?

Trả lời mẫu: App không chỉ đọc một lần. Với FastALPR, app đọc crop gốc, đọc thêm bản ảnh được tăng tương phản/sharpen và thử đọc biển hai dòng bằng cách tách crop thành phần trên/dưới. Sau đó app chọn candidate tốt nhất.

5.7. FastPlateOCRWrapper

Wrapper này là backend dự phòng. Nó dùng LicensePlateRecognizer và gọi run_one. Điểm khác:

- Chuyển ảnh BGR sang RGB trước khi OCR.
- Nếu có char_probs thì lấy trung bình làm confidence.
- Có thêm candidate sau tiền xử lý Otsu trong preprocess_plate_debug.

Ý cần học: Dù backend khác nhau, app vẫn giữ interface read(crop) giống nhau, nên phần detect và GUI không cần biết OCR cụ thể là thư viện nào.

6. GUI Và Điều Phối Chạy

6.1. App.__init__

App kế thừa tk.Tk, nên nó là cửa sổ chính của chương trình. Trong __init__, app làm các việc:

- đặt title, kích thước cửa sổ, màu nền;
- tạo các biến trạng thái: model, OCR, worker thread, cancel flag;
- tạo các StringVar, DoubleVar, BooleanVar để liên kết GUI với giá trị;
- lấy preset mặc định Balanced;
- tìm FFmpeg;
- gọi configure_style, build_ui, load_config.

6.2. Các biến quan trọng trong __init__

Biến	Ý nghĩa
self.model	YOLO model đã nạp. Nếu chưa nạp thì là None.
self.ocr	OCR backend đã nạp.
self.loaded_model_path	Đường dẫn model hiện đang nạp, để tránh nạp lại nếu không đổi model.
self.cancel_flag	Cờ dừng khi người dùng bấm STOP.
self.worker	Thread chạy xử lý ảnh/video để GUI không bị treo.
self.rows	Dữ liệu kết quả sẽ ghi ra CSV.
self.preset_var	Preset đang chọn.
self.det_conf_var	Ngưỡng YOLO confidence trên GUI.
self.trust_var	Ngưỡng reliability tối thiểu để hiển thị.
self.ffmpeg_path	Đường dẫn FFmpeg dùng xử lý video.

Bảng 4: Các biến trạng thái quan trọng của App

6.3. build_ui

build_ui tạo toàn bộ giao diện:

- Header: tên app HCMUS VN Plate OCR.
- Panel chọn model và input.
- Combobox chọn profile.
- Slider chỉnh YOLO confidence và min trust.
- Nút RUN/STOP.
- Advanced panel: encoder, OCR backend, max plates, imgsz, crop pad, các checkbox.

- Canvas preview ảnh/video.
- Table results.
- Crop preview.
- Progress bar và status.

Ý cần nói: GUI không xử lý AI trực tiếp; GUI chỉ lấy tham số và hiển thị. Phần AI nằm ở detect_frame, OCR wrapper và YOLO model.

6.4. cfg

cfg() lấy preset đang chọn rồi ghi đè bằng các giá trị người dùng chỉnh trên GUI. Đây là hàm quan trọng vì mọi xử lý sau đó dùng cfg chứ không đọc trực tiếp từng biến GUI.

Ví dụ:

- cfg["conf"] từ slider YOLO confidence.
- cfg["min_reliability"] từ slider min trust.
- cfg["imgsz"] từ slider image size.
- cfg["tile_fallback"] bật khi Heavy scan.

Câu hỏi: Vì sao có hàm cfg()?

Trả lời mẫu: Vì app có preset nhưng người dùng vẫn có thể chỉnh tay trên GUI. Hàm cfg() gom preset và các giá trị chỉnh tay thành một dictionary thống nhất để các hàm detect, video, filter dùng cùng một cấu hình.

6.5. load_model và load_ocr

load_model kiểm tra thư viện Ultralytics, kiểm tra file .pt, rồi nạp YOLO bằng YOLO(str(p)). Nếu model đã nạp và đường dẫn không đổi, app dùng lại model cũ để tiết kiệm thời gian.

load_ocr chọn backend OCR:

- Nếu Auto và có FastALPR thì dùng FastALPR DefaultOCR.
- Nếu không có FastALPR nhưng có fast_plate_ocr thì dùng backend đó.
- Nếu người dùng chọn cụ thể thì nạp backend tương ứng.

6.6. start và run

start không chạy xử lý trực tiếp trên GUI thread. Nó tạo:

```
self.worker = threading.Thread(target=self.run, daemon=True)
self.worker.start()
```

Lý do: xử lý ảnh/video lâu, nếu chạy ngay trong main thread thì cửa sổ Tkinter bị đơ. Thread riêng giúp GUI vẫn phản hồi, có thể update progress hoặc bấm STOP.

run là hàm điều phối:

1. Lưu config.
2. Kiểm tra input tồn tại.
3. Nạp YOLO model.
4. Nạp OCR backend.
5. Nếu input là ảnh/folder ảnh thì gọi `run_images`.
6. Nếu input là video thì gọi `run_video`.
7. Ghi `self.rows` ra CSV.
8. Bắt lỗi và hiển thị messagebox nếu fail.

7. Chế Độ Xử Lý Ảnh

7.1. collect_images

Hàm này gom danh sách ảnh cần xử lý:

- Nếu input là một file ảnh, trả về danh sách có một ảnh.
- Nếu input là thư mục, quét đệ quy tất cả file có đuôi ảnh.
- Nếu không hợp lệ thì báo lỗi.

7.2. run_images

Luồng run_images:

1. Gọi collect_images.
2. Lấy cấu hình bằng cfg().
3. Chọn device YOLO bằng resolve_yolo_device.
4. Duyệt từng ảnh.
5. Đọc ảnh bằng cv2.imread.
6. Gọi detect_frame để YOLO + OCR.
7. Gọi clean_display_results để lọc kết quả.
8. Gọi log_image_results để đưa vào bảng/CSV.
9. Gọi draw để vẽ box/text.
10. Lưu ảnh vào annotated_images.
11. Cập nhật preview và progress.

Câu hỏi: Ảnh đơn và thư mục ảnh chạy khác nhau không?

Trả lời mẫu: Về xử lý AI thì giống nhau. Khác ở bước collect_images: ảnh đơn tạo danh sách một phần tử, thư mục ảnh thì quét nhiều ảnh. Sau đó mỗi ảnh đều đi qua detect_frame, clean_display_results, draw và ghi output.

7.3. log_image_results

Hàm này chuyển PlateResult thành row CSV. Mỗi row gồm:

- input name;
- plate text;
- reliability;
- detection confidence;
- OCR confidence;
- source;

- bbox;
- crop path.

Ý nghĩa: CSV giúp chứng minh app không chỉ vẽ ảnh, mà còn xuất dữ liệu có thể kiểm tra lại.

8. Chế Độ Xử Lý Video

8.1. Vì sao video phức tạp hơn ảnh?

Ảnh là một frame độc lập. Video là chuỗi nhiều frame, nên nếu OCR từng frame riêng lẻ thì kết quả dễ nhấp nháy. Có frame rõ, có frame mờ, có frame YOLO bỏ sót. Vì vậy app có thêm tracking, hold frame, OCR voting và FFmpeg.

8.2. run_video tổng quan

Luồng run_video:

1. Tìm ffmpeg.exe và ffprobe.exe.
2. Dùng ffprobe đọc width, height, fps, total frames, codec.
3. Tính stride từ $\text{fps} / \text{sample_fps}$.
4. Tạo lệnh FFmpeg decode video thành raw BGR frames.
5. Tạo lệnh FFmpeg encode raw frames đã annotate thành video mp4.
6. Duyệt từng frame raw.
7. Mỗi stride frame mới chạy YOLO + OCR.
8. Cập nhật track và OCR voting.
9. Chuyển track thành kết quả hiển thị.
10. Vẽ annotation và gửi frame vào FFmpeg encoder.
11. Sau khi xong, mux audio gốc vào video output.

8.3. sample_fps và stride

```
sample_fps = max(0.1, float(cfg["sample_fps"]))
stride = max(1, int(round(fps / sample_fps)))
```

Nếu video 30 fps và $\text{sample_fps} = 10$, app không cần chạy YOLO cả 30 frame/giây. Nó chạy khoảng mỗi 3 frame một lần. Điều này tăng tốc, còn các frame giữa vẫn dùng track cũ để vẽ.

Câu hỏi: Vì sao video không detect mọi frame?

Trả lời mẫu: Detect mọi frame rất tốn thời gian. App dùng sample_fps để chỉ chạy YOLO trên một số frame, sau đó dùng tracking và hold frame để giữ kết quả trên các frame còn lại. Cách này cân bằng tốc độ và độ ổn định khi demo.

8.4. FFmpeg decode/encode

App dùng FFmpeg theo kiểu pipe:

- Decoder đọc video input, xuất rawvideo bgr24 ra stdout.
- Python đọc từng frame raw từ stdout.
- Python vẽ annotation.
- Encoder nhận frame đã vẽ qua stdin và tạo video mp4.

Lý do dùng bgr24: OpenCV dùng ảnh BGR, nên nhận frame trực tiếp ở dạng BGR giúp giảm chuyển đổi.

8.5. choose_video_encoder

Hàm này chọn encoder video:

- Auto ưu tiên NVIDIA NVENC nếu có.
- Nếu không có thì thử Intel QuickSync.
- Nếu không có thì thử AMD AMF.
- Cuối cùng fallback CPU x264.

Nếu encoder được chọn không có trong FFmpeg hiện tại, app tự fallback về CPU x264. Đây là xử lý an toàn để tránh demo bị fail vì máy không có GPU/driver phù hợp.

8.6. mux_audio_from_source

Sau khi encode video annotated, video tạm thời chưa có audio. mux_audio_from_source copy video stream đã annotate và lấy audio từ video gốc. Nếu video gốc không có audio hoặc mux lỗi, app vẫn giữ video không tiếng thay vì fail toàn bộ.

9. Hàm `detect_frame` - Trái Tim Của App

9.1. Vai trò

`detect_frame` là hàm quan trọng nhất vì nó nối YOLO và OCR. Cả ảnh và video đều đi qua hàm này. Có thể xem nó là pipeline mini cho một ảnh/frame.

9.2. Luồng xử lý của `detect_frame`

1. Lấy kích thước frame.
2. Tạo danh sách `raw_boxes`.
3. Gọi `make_inference_variants` để tạo ảnh gốc, ảnh sáng hơn, ảnh xoay, ảnh split, tile.
4. Chạy YOLO trên từng variant bằng `predict_boxes_on_image`.
5. Map box từ variant về tọa độ ảnh gốc.
6. Gộp box trùng bằng `dedup_detection_boxes`.
7. Nếu bật fallback, thêm box từ `classical_plate_fallback_boxes`.
8. Lọc hình học bằng `pass_geometry_filter`.
9. Crop biển số bằng `crop_box`.
10. Lưu crop vào `CROP_DIR`.
11. Gọi `ocr.read(crop)`.
12. Nếu OCR rỗng thì đặt `UNKNOWN`.
13. Ghép source với tên variant.
14. Chặn nhãn nền bằng `reject_by_anti_sign_guard`.
15. Tạo `PlateResult`.
16. Nếu `log_rows=True`, ghi row vào bảng/CSV.

Câu học thuộc về `detect_frame`

“`detect_frame` là hàm lõi. Nó tạo nhiều biến thể ảnh để YOLO dễ tìm biển, chạy YOLO, dedup box, crop biển số, OCR crop, lọc mã nền bãi xe, rồi trả về danh sách `PlateResult` để vẽ hoặc tracking.”

9.3. `make_inference_variants`

Hàm này tạo nhiều phiên bản của cùng một frame để tăng khả năng phát hiện:

- `orig`: ảnh gốc.
- `gamma065`: ảnh làm sáng cho bãi xe tối.
- `rot_cw90`, `rot_ccw90`: xoay 90 độ để cứu biển nghiêng/dọc.
- `split_left`, `split_right`, `split_top`, `split_bottom`: chia ảnh đông xe.

- `tile_i_j`: chia grid có overlap để tìm biển nhỏ.

Mỗi variant đi kèm một hàm mapper để chuyển bounding box từ ảnh variant về tọa độ ảnh gốc. Đây là điểm quan trọng: nếu YOLO detect trên tile, box phải cộng offset tile mới vẽ đúng trên ảnh gốc.

9.4. `predict_boxes_on_image`

Hàm này gọi YOLO:

```
model.predict(img, imgsz=imgsz, conf=conf, iou=0.42, max_det=60,
device=device)
```

Nó lấy xxyy và confidence của từng box, clamp tọa độ vào kích thước ảnh rồi trả về danh sách (`bbox`, `conf`).

9.5. `dedup_detection_boxes`

Vì app chạy YOLO trên nhiều variant, cùng một biển số có thể bị detect nhiều lần. `dedup_detection_boxes` sắp xếp box theo confidence giảm dần, rồi giữ box nếu IoU với các box đã giữ nhỏ hơn ngưỡng. Đây là Non-Maximum Suppression ở mức app.

9.6. `pass_geometry_filter`

Hàm này lọc box bất thường theo:

- area ratio: box không quá nhỏ, không quá lớn so với ảnh;
- aspect ratio: tỷ lệ rộng/cao phải nằm trong khoảng hợp lý.

Mục đích là loại vùng tường, cột, bảng chữ hoặc vùng quá lớn không giống biển số.

9.7. `classical_plate_fallback_boxes`

Đây là fallback xử lý ảnh cổ điển khi YOLO bỏ sót. Nó dùng HSV, grayscale, Canny edge, morphology và contour để đề xuất box có hình dạng giống biển số.

Ý cần nói: fallback không thay thế YOLO. Nó chỉ hỗ trợ trong ca khó khi YOLO miss, nhất là biển dài, sáng hoặc có biên rõ.

9.8. `reject_by_anti_sign_guard`

Hàm này chặn false positive trong bãi xe. Nó kết hợp:

- text OCR có giống mã cột/ô đậu không;
- visual score của crop có giống biển thật không;
- confidence YOLO;
- reliability score;
- nguồn detection có phải fallback dễ nhầm không.

Ví dụ D28, D29 dễ bị OCR nhầm thành biển. Anti-sign guard giúp không hiển thị các kết quả đó.

10. Tracking Và OCR Voting Trong Video

10.1. Vì sao cần tracking?

Trong video, cùng một biển số xuất hiện qua nhiều frame. Nếu xử lý từng frame độc lập, text có thể lúc đúng lúc sai. Tracking giúp gom các detection gần nhau theo thời gian thành một track, giữ box mượt hơn và chọn OCR tốt nhất qua nhiều frame.

10.2. bbox_iou và ema_box

bbox_iou đo mức trùng nhau giữa box mới và box track cũ. Nếu IoU đủ lớn, app xem đó là cùng một biển.

ema_box làm mượt box:

$$\text{smooth_bbox} = \text{old} * (1 - a) + \text{new} * a$$

Nếu smooth thấp, box thay đổi chậm và ổn định hơn. Nếu smooth cao, box bám nhanh theo detection mới.

10.3. add_ocr_vote

Hàm này cộng phiếu cho text OCR trong một track. Text hợp lệ được cộng trọng số cao hơn bằng plate_vote_weight. Track lưu:

- votes[text]["score"]
- best_conf
- count

Text có điểm vote cao nhất trở thành best_text.

10.4. update_tracks

Luồng update_tracks:

1. Sắp xếp detections theo confidence.
2. Với mỗi detection, tìm track cũ có IoU cao nhất.
3. Nếu IoU đủ lớn, cập nhật track:
 - bbox mới;
 - smooth bbox;
 - last seen;
 - hits;
 - det confidence trung bình;
 - crop path;
 - OCR vote.

4. Detection không match thì tạo track mới.
5. Xóa track quá cũ.
6. Nếu track đủ tin cậy thì ghi row CSV.

10.5. tracks_to_results

Hàm này chuyển track đang còn hiệu lực thành PlateResult để vẽ lên frame. Nó dùng age_factor để giảm confidence nếu track đã lâu không thấy lại. Sau đó gọi clean_display_results để lọc lần cuối.

Câu hỏi: OCR voting trong video dùng để làm gì?

Trả lời mẫu: Vì OCR ở từng frame có thể dao động, app gom nhiều lần đọc trong cùng một track và cộng điểm cho chuỗi ổn định hơn. Chuỗi hợp lệ và có OCR confidence cao sẽ thắng vote, giúp text trên video ít nhấp nháy hơn.

11. Lọc Kết Quả Và Vẽ Output

11.1. `clean_display_results`

Hàm này lọc danh sách `PlateResult` trước khi vẽ:

- Ẩn UNKNOWN nếu cấu hình yêu cầu.
- Chặn mã nền bãi xe bằng `looks_like_parking_area_code`.
- Nếu bật `only_valid_vn`, chỉ giữ chuỗi giống biển số.
- Tính reliability và loại kết quả dưới `min_reliability`.
- Sắp xếp theo valid plate, OCR conf, det conf và area.
- Gọi `suppress_same_region` để loại kết quả trùng vùng.
- Giới hạn số kết quả bằng `max_show`.

11.2. `draw`

`draw` nhận frame và danh sách `PlateResult`, sau đó:

1. Lọc lại kết quả.
2. Copy frame.
3. Với mỗi biển, lấy bbox.
4. Tạo label gồm text và reliability.
5. Vẽ rectangle màu xanh.
6. Tính kích thước chữ để không quá dài.
7. Vẽ chữ có outline đen và foreground trắng.
8. Trả về frame đã annotate.

Ý cần nói: `draw` chỉ hiển thị kết quả, không quyết định AI. Quyết định AI nằm ở `detect/OCR/filter` trước đó.

11.3. `show_frame` và `show_crop_preview`

`show_frame` chuyển BGR sang RGB, dùng Pillow resize ảnh cho vừa canvas, rồi hiển thị lên Tkinter.

`show_crop_preview` hiển thị crop biển số khi người dùng chọn một row trong bảng kết quả. Nó cũng in label gồm plate, trust, det và OCR confidence.

12. Bảng Học Từng Hàm Chính

12.1. Nhóm xử lý biển số và box

Hàm	Giải thích để học
<code>clean_text</code>	Chuẩn hóa chuỗi về chữ hoa và chỉ giữ chữ/số.
<code>fix_digits</code>	Sửa ký tự dễ nhầm sang số trong đoạn cần số.
<code>fix_letters</code>	Sửa số dễ nhầm sang chữ trong đoạn cần chữ.
<code>valid_plate</code>	Kiểm tra chuỗi có đúng regex biển số Việt Nam và mã tỉnh hợp lệ.
<code>old_numeric_plate</code>	Chấp nhận dạng biển số số cũ/loose như chuỗi 7-8 số.
<code>plausible_plate_text</code>	Chuỗi có vẻ là biển số nếu valid hoặc dạng số cũ.
<code>looks_like_parking_area_code</code>	Chặn mã nền/cột bãi xe như D28, D29.
<code>normalize_plate</code>	Tạo candidate biển số từ OCR thô, sửa chữ/số theo ngữ cảnh và chọn candidate tốt nhất.
<code>bbox_iou</code>	Tính IoU để dedup box và match track trong video.
<code>ema_box</code>	Làm mượt box qua thời gian trong video.
<code>plate_vote_weight</code>	Tính trọng số vote OCR trong video.
<code>expand_box</code>	Nới box trước khi crop để tránh mất ký tự.
<code>crop_box</code>	Cắt vùng biển số từ ảnh theo box.
<code>reliability_score</code>	Tính điểm tin cậy tổng hợp từ OCR conf, YOLO conf và tính hợp lệ của chuỗi.

Bảng 5: Các hàm tiện ích cần học

12.2. Nhóm OCR và app

Hàm / lớp	Giải thích để học
PlateResult	Dataclass chứa bbox, det conf, text, display, OCR conf, source, crop path.
FastALPROCR	Wrapper OCR chính dùng FastALPR DefaultOCR.
FastALPROCR.read	Đọc crop gốc, crop enhance, candidate hai dòng và chọn kết quả tốt nhất.
preprocess_plate_debug	Tiền xử lý ảnh xám, equalize, resize, blur, Otsu cho backend phụ.
FastPlateOCRWrapper	Wrapper backend dự phòng fast_plate_ocr.
App.__init__	Khởi tạo cửa sổ, biến trạng thái, preset, FFmpeg, UI và config.
build_ui	Tạo giao diện: chọn model/input, profile, advanced, preview, results, crop preview.
cfg	Gom preset và giá trị GUI thành cấu hình chạy.
load_model	Nạp YOLO .pt bằng Ultralytics.
load_ocr	Chọn và nạp OCR backend.
start	Tạo worker thread để chạy mà GUI không treo.
run	Điều phối toàn bộ: nạp model/OCR, rẽ nhánh ảnh hoặc video, ghi CSV.

Bảng 6: Các lớp/hàm điều phối chính

12.3. Nhóm xử lý ảnh/video

Hàm	Giải thích để học
run_images	Xử lý ảnh đơn hoặc thư mục ảnh, gọi detect_frame, vẽ ảnh, lưu output.
run_video	Xử lý video bằng FFmpeg pipe, sampling, tracking, OCR voting, encode output.
choose_video_encoder	Chọn encoder NVENC/QSV/AMF/x264 và fallback an toàn.
make_inference_variants	Tạo ảnh gốc, gamma, xoay, split, tile để YOLO tìm biển tốt hơn.
predict_boxes_on_image	Gọi model.predict và lấy box/conf.
dedup_detection_boxes	Loại box trùng từ nhiều variant.
pass_geometry_filter	Lọc box quá lạ theo area ratio và aspect ratio.
classical_plate_fallback_boxes	Fallback bằng xử lý ảnh cổ điển khi YOLO miss.
detect_frame	Hàm lõi nối YOLO, crop, OCR, guard, PlateResult.
update_tracks	Match detection với track cũ, làm mượt box, cộng OCR vote.
tracks_to_results	Chuyển track thành kết quả hiển thị trên frame.
clean_display_results	Lọc kết quả cuối, chống trùng vùng, giới hạn số biển hiển thị.
draw	Vẽ bounding box và nhãn lên ảnh/video.

Bảng 7: Các hàm xử lý ảnh/video quan trọng

13. Câu Hỏi Vắn Đáp Về App Python

13.1. Câu hỏi tổng quan

Câu hỏi: App Python khác notebook ở điểm nào?

Trả lời mẫu: Notebook dùng để huấn luyện và đánh giá mô hình, còn app Python dùng để triển khai demo. App nạp model đã train, nhận ảnh/video từ người dùng, chạy YOLO + OCR, hiển thị kết quả và lưu output/CSV.

Câu hỏi: Luồng xử lý chính của app là gì?

Trả lời mẫu: Người dùng chọn model và input. App nạp YOLO, nạp OCR, nếu input là ảnh thì chạy `run_images`, nếu là video thì chạy `run_video`. Mỗi ảnh/frame đi qua `detect_frame`: tạo variant, YOLO detect, dedup box, crop biển số, OCR, lọc nhiễu, tạo `PlateResult`, sau đó vẽ và lưu kết quả.

13.2. Câu hỏi về YOLO trong app

Câu hỏi: YOLO được gọi ở đâu?

Trả lời mẫu: YOLO được nạp trong `load_model` bằng `YOLO(str(p))`. Khi suy luận, app gọi YOLO trong `predict_boxes_on_image` bằng `model.predict(...)`. Hàm `detect_frame` điều phối việc gọi YOLO trên ảnh gốc và các variant.

Câu hỏi: Vì sao app chạy YOLO trên nhiều variant?

Trả lời mẫu: Vì ảnh thực tế có thể tối, nghiêng, rộng hoặc có nhiều biển nhỏ. App tạo variant như ảnh gamma sáng hơn, ảnh xoay, split và tile để tăng cơ hội phát hiện. Sau đó box từ variant được map về ảnh gốc và dedup.

13.3. Câu hỏi về OCR

Câu hỏi: OCR được gọi ở đâu?

Trả lời mẫu: OCR được nạp trong `load_ocr`. Khi đã có crop biển số, `detect_frame` gọi `ocr.read(crop)`. Tùy backend, `read` có thể là của `FastALPROCR` hoặc `FastPlateOCRWrapper`.

Câu hỏi: Tại sao OCR có source?

Trả lời mẫu: source ghi lại kết quả đến từ backend nào và biến thể nào, ví dụ crop gốc, crop sharpen, two-line hoặc variant YOLO. Nó giúp debug khi kết quả sai và giúp báo cáo rõ nguồn kết quả.

13.4. Câu hỏi về video

Câu hỏi: Vì sao video cần tracking?

Trả lời mẫu: Vì cùng một biển xuất hiện qua nhiều frame và OCR có thể dao động. Tracking gom các detection gần nhau theo IoU, làm mượt box và dùng OCR voting để chọn text ổn định hơn.

Câu hỏi: FFmpeg dùng để làm gì trong app?

Trả lời mẫu: FFmpeg dùng để decode video thành frame BGR cho OpenCV xử lý, sau đó encode lại frame đã annotate thành video mp4. Cuối cùng app mux audio gốc vào video output nếu có.

13.5. Câu hỏi về lỗi và giới hạn

Câu hỏi: Nếu app nhận nhầm D28/D29 thành biển số thì xử lý ở đâu?

Trả lời mẫu: App có `looks_like_parking_area_code` và `reject_by_anti_sign_guard`. Hai phần này kiểm tra chuỗi OCR, mã tỉnh, visual score của crop, confidence và source để chặn các mã nền bãi xe như D28, D29.

Câu hỏi: Nếu YOLO bỏ sót biển số thì app có cách cứu không?

Trả lời mẫu: Có. App có thể giảm confidence bằng preset, bật heavy scan để split/tile ảnh, chạy ảnh gamma/rotate, và dùng `classical_plate_fallback_boxes` để đề xuất box bằng xử lý ảnh cổ điển khi YOLO miss.

Câu hỏi: Nếu thầy hỏi phần nào em cần chỉ trên code trước?

Trả lời mẫu: Em nên chỉ theo thứ tự: `load_model` để thấy YOLO được nạp, `load_ocr` để thấy OCR backend, `run` để thấy rõ nhánh ảnh/video, `detect_frame` để thấy YOLO-crop-OCR, `run_video` để thấy FFmpeg/tracking, và `draw` để thấy kết quả được vẽ.

14. Kịch Bản Mở Code Khi Vấn Đáp

14.1. Thứ tự trình bày khi mở file app

1. Mở phần import và requirements để nói app dùng Ultralytics, OpenCV, OCR, Tkinter, FFmpeg.
2. Mở PRESETS để nói app có nhiều chế độ chạy theo tình huống.
3. Mở normalize_plate, valid_plate, reliability_score để nói cách chuẩn hóa/lọc kết quả.
4. Mở FastALPROMCR.read để nói OCR đọc crop gốc, crop enhance, biến hai dòng.
5. Mở App.__init__ và build_ui để nói app là GUI desktop.
6. Mở load_model, load_ocr, start, run để nói luồng điều phối.
7. Mở run_images để nói chế độ ảnh.
8. Mở run_video để nói chế độ video, FFmpeg, stride.
9. Mở detect_frame để nói hàm lõi YOLO + OCR.
10. Mở update_tracks và add_ocr_vote để nói ổn định kết quả video.
11. Mở draw và log_image_results để nói xuất ảnh/video và CSV.

14.2. Đoạn nói mẫu 1 phút

App Python của nhóm là công cụ demo cho pipeline nhận dạng biển số. Về kỹ thuật, app có ba phần chính. Phần thứ nhất là GUI Tkinter để chọn model, input và preset. Phần thứ hai là pipeline AI trong detect_frame, gồm tạo biến thể ảnh, YOLO detect, dedup box, crop biển số, OCR và lọc kết quả nhiễu. Phần thứ ba là output: ảnh/video annotated, crop biển số, bảng kết quả và CSV. Với video, app dùng thêm FFmpeg để đọc/ghi frame và dùng tracking/OCR voting để giảm hiện tượng text nhấp nháy giữa các frame.

15. Kết Luận Học Thuộc

15.1. Ba câu quan trọng nhất

Câu 1

`detect_frame` là hàm lõi của app vì nó nối YOLO với OCR: detect box, crop biển, OCR, lọc nhiễu và trả về `PlateResult`.

Câu 2

App có nhiều preset và inference variants vì ảnh/video thực tế có nhiều tình huống khó: tối, nghiêng, biển nhỏ, nhiều xe, video rung.

Câu 3

Video cần tracking và OCR voting vì đọc từng frame độc lập dễ nhấp nháy; gom nhiều frame giúp kết quả ổn định hơn.